



TheDBAdmin.com

Practitioner Reference · Training Companion

PostgreSQL Performance Tuning

Tools · SQL Tuning · EXPLAIN · Cloud (RDS / Azure) · Hands-On Labs

© TheDBAdmin.com · All Rights Reserved

About This Guide

This guide is a concise field reference for diagnosing and tuning PostgreSQL workloads on self-managed and cloud-managed deployments. It assumes familiarity with PostgreSQL architecture (shared buffers, WAL, MVCC, autovacuum) and SQL. Each section is action-oriented: what to look at, which tool, and what the signal means. The companion labs at the end of this guide turn each tool into a hands-on exercise students can complete in a sandbox.

How to use this document. Sections 1–9 are the reference. Section 10 is a self-contained lab manual that can be run in order against a local PostgreSQL 15+ sandbox seeded with `pgbench`. Each lab declares its objective, prerequisites, steps, and discussion prompts so it can be lifted into a workshop or self-study track.

1. Tuning Methodology

Effective tuning follows a top-down loop: **Workload** → **Query** → **Plan** → **Server** → **OS / Storage**. Optimize where the time is actually spent. Without measurement, every change is a guess.

- **Workload baseline:** TPS, active sessions, wait events, peak vs steady state.
- **Query profile:** Top-N by total exec time, mean time, calls, I/O, and rows returned.
- **Plan analysis:** Row estimate accuracy, join order, access methods, buffers.
- **Server:** Memory parameters, autovacuum, checkpoints, connection pooling.
- **OS / Storage:** `iowait`, page cache hit ratio, throughput vs IOPS limits, NUMA, transparent huge pages.

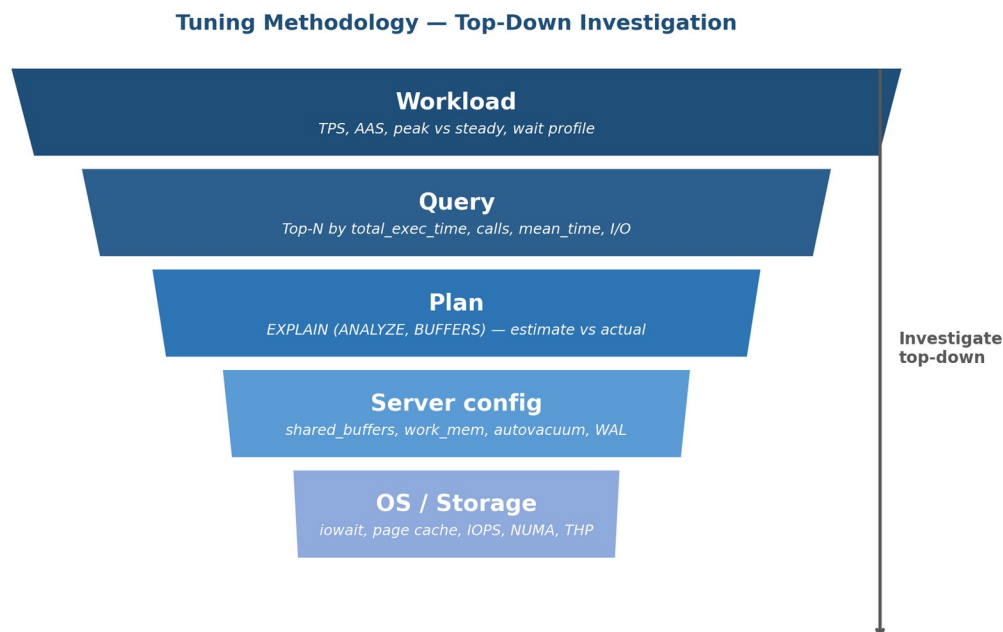


Figure 1 — Top-down tuning investigation. Start at the workload layer; only descend when you have evidence the layer above is healthy.

2. Core Server Parameters

These are the parameters with the highest leverage on most workloads. Always change one knob, observe, and document.

Parameter	Typical Start	Purpose & Tuning Notes
<code>shared_buffers</code>	25% of RAM	PostgreSQL's own page cache. Above 40% rarely helps; the OS page cache also caches pages.
<code>effective_cache_size</code>	50–75% of RAM	Planner hint, not an allocation. Helps the planner choose index scans over seq scans.
<code>work_mem</code>	16–64 MB	Per sort/hash operation, per query, per parallel worker. Excessive values cause OOM.
<code>maintenance_work_mem</code>	512 MB – 2 GB	VACUUM, CREATE INDEX, ALTER TABLE. Bigger = faster maintenance.
<code>random_page_cost</code>	1.1 (SSD/cloud)	Default 4.0 is HDD-era. On SSD/EBS gp3/NVMe set to 1.1–1.5.
<code>effective_io_concurrency</code>	200 (SSD)	Bitmap heap scan prefetch. 1–2 for HDD, 200+ for SSD/cloud block storage.
<code>max_wal_size</code>	4–16 GB	Larger = fewer checkpoints, smoother writes, more crash recovery time.
<code>checkpoint_timeout</code>	15 min	Pair with <code>max_wal_size</code> . Watch for 'checkpoints occurring too frequently' in logs.
<code>max_connections</code>	100–300	Use PgBouncer instead of raising this. Each connection is a process with memory cost.
<code>autovacuum*</code>	Aggressive	Lower <code>scale_factor</code> (0.05) and <code>cost_delay</code> for write-heavy tables.

Rule of thumb

Sizing parameters scale with RAM. Behavior parameters (`random_page_cost`, `autovacuum`) scale with workload shape. Don't copy production values into a 4 GB dev box.

3. Diagnostic Tools — The Classics

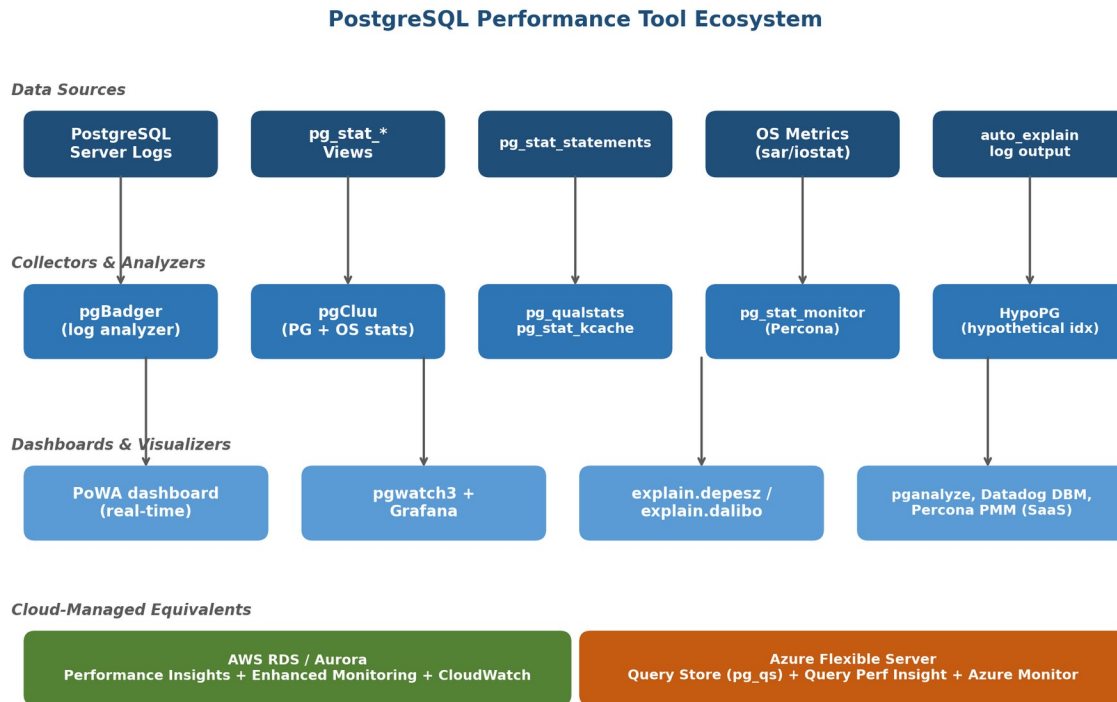


Figure 2 — Tool ecosystem. Data sources at the top feed into collectors, which in turn drive dashboards. Cloud-managed services collapse several of these layers into a single product.

3.1 pg_stat_statements (built-in extension)

The single most important extension. Aggregates normalized query statistics — total time, mean time, calls, rows, I/O. Enable in every environment.

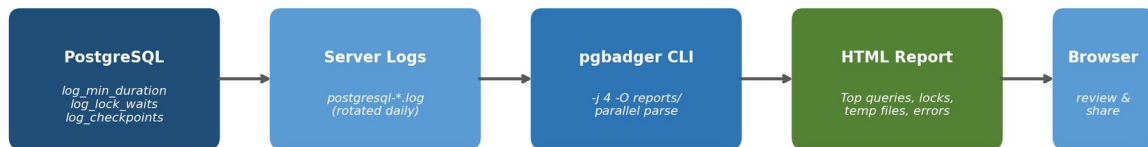
```

-- postgresql.conf
shared_preload_libraries = 'pg_stat_statements'
pg_stat_statements.track = all
pg_stat_statements.max = 10000

-- Top 10 queries by total execution time
SELECT substring(query, 1, 80) AS query,
       calls, total_exec_time::int AS total_ms,
       mean_exec_time::int AS mean_ms,
       rows, shared_blks_hit, shared_blks_read
FROM   pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
  
```

3.2 pgBadger — log analyzer

A Perl-based offline log parser that turns `postgresql.log` into a self-contained HTML report: top queries by time, slowest queries, error distribution, lock waits, temp file usage, connection histograms, and checkpoint timing.

pgBadger — Log-to-Report Workflow

Tip: For high-volume systems, run pgbadger incrementally with --incremental and --outdir; schedule via cron on a non-prod host.

Figure 3 — pgBadger pipeline. The quality of the report is determined by the `log_line_prefix` and the verbosity of logging on the server.

```
# Minimum logging for pgBadger:
log_min_duration_statement = 500
log_line_prefix = '%t [%p]: db=%d,user=%u,app=%a,client=%h '
log_checkpoints = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0

# Generate report:
pgbadger -j 4 -O reports/ /var/log/postgresql/*.log
```

3.3 pgCluu — full-stack stats collector

A two-part toolkit: `pgcluu_collectedd` gathers PostgreSQL + system metrics (sar, iostat, vmstat) on a schedule; `pgcluu` builds an HTML report. Useful where you want PG and OS metrics correlated without a Prometheus stack.

3.4 auto_explain

Automatically logs the EXPLAIN plan for queries exceeding a duration threshold. Critical for catching plan changes you would otherwise miss.

```
shared_preload_libraries = 'auto_explain, pg_stat_statements'
auto_explain.log_min_duration = '1s'
auto_explain.log_analyze = on          -- runtime stats (adds overhead)
auto_explain.log_buffers = on
auto_explain.log_timing = off          -- reduces overhead on Linux
auto_explain.log_nested_statements = on
auto_explain.sample_rate = 0.1        -- 10% sampling for high QPS
```

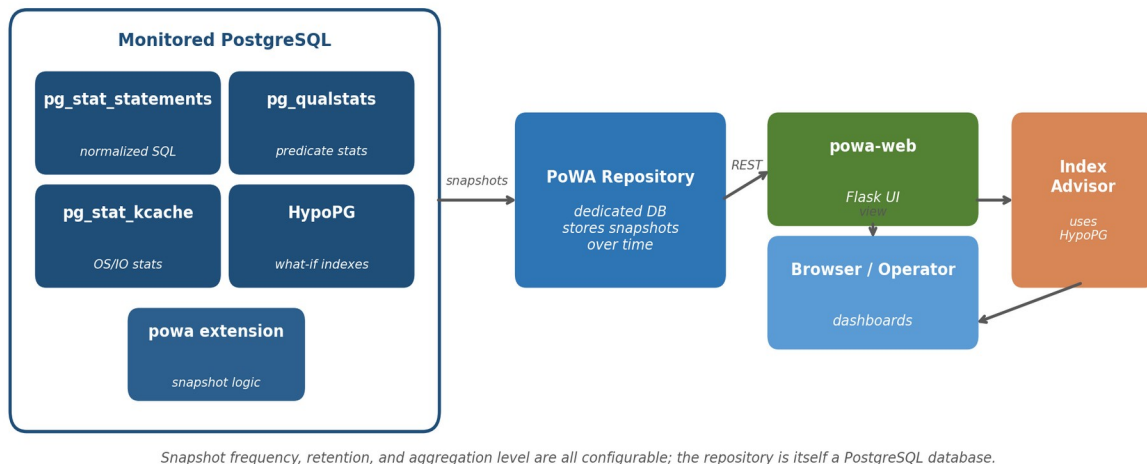
3.5 pgbench

The built-in benchmark. Useful less for synthetic TPC-B numbers and more for replaying custom scripts against a tuned vs untuned configuration on identical hardware.

4. Diagnostic Tools — Modern Stack

Tool	What It Adds	When To Use
PoWA	Real-time dashboard built on pg_stat_statements + pg_qualstats + pg_stat_kcache + HypoPG	Continuous workload analysis with index recommendations.
pg_qualstats	Per-predicate selectivity statistics — which WHERE clauses are run, with what filters	Finding missing indexes that pg_stat_statements alone can't see.
pg_stat_kcache	Per-query OS-level CPU and physical disk reads via getrusage	Distinguishing CPU-bound from I/O-bound queries.
HypoPG	Hypothetical indexes — test EXPLAIN with a virtual index before creating it	Index design on large tables where CREATE INDEX is expensive.
pg_profile / pgpro_pwr	AWR-style historical snapshot reports	Postmortems and trend analysis without a TSDB.
pgwatch2 / pgwatch3	Grafana dashboards over InfluxDB / Postgres / Prometheus	Long-term monitoring with alerting.
pganalyze, Datadog DBM, Percona PMM	Commercial / SaaS observability with plan history and index advisors	When you need plan-change detection and ML-driven advice out of the box.
pg_stat_monitor (Percona)	Bucketed pg_stat_statements with histograms and client info	When you need time-windowed query stats rather than cumulative.

PoWA — PostgreSQL Workload Analyzer Architecture



Snapshot frequency, retention, and aggregation level are all configurable; the repository is itself a PostgreSQL database.

Figure 4 — PoWA architecture. The monitored database hosts the source extensions; PoWA snapshots them into a separate repository that `powa-web` reads for dashboards and index recommendations.

Stack recommendation

For a self-managed deployment in 2026, a strong baseline is: `pg_stat_statements` + `auto_explain` + `pg_qualstats` + `pg_stat_kcache`, scraped by `pgwatch3/Prometheus`, with PoWA for ad-hoc deep dives and `HypoPG` for index design.

5. SQL Tuning — Patterns and Anti-patterns

5.1 Sargability

A predicate is *sargable* if the planner can use an index on the column. Wrapping the indexed column in a function destroys sargability.

```
-- NOT sargable: function on the indexed column
WHERE lower(email) = 'a@b.com'
WHERE date_trunc('day', created_at) = '2026-05-14'

-- Sargable equivalents
WHERE email = lower('A@B.com')           -- or use a functional index
WHERE created_at >= '2026-05-14' AND created_at < '2026-05-15'
```

5.2 Selectivity, cardinality, and the planner

- The planner picks scan and join methods using row estimates from `pg_statistic`. Bad estimates = bad plans.
- Increase `default_statistics_target` (100 → 500–1000) on columns with skewed distributions, or use `ALTER TABLE ... ALTER COLUMN ... SET STATISTICS`.

- For correlated columns, use `CREATE STATISTICS` (dependencies, ndistinct, mcv) — often forgotten.
- Always run `ANALYZE` after bulk loads; autovacuum may not have caught up.

5.3 Join strategies

Strategy	Best When	Failure Mode
Nested Loop	Inner side has an index and outer side is small (<10k rows).	Outer side balloons due to underestimation → quadratic blow-up.
Hash Join	Equality join, one side fits in work_mem.	Hash table spills to disk; raise work_mem or reduce build side.
Merge Join	Both sides already sorted (e.g., on indexed columns).	Sort spills to disk; usually a sign of a missing index.

5.4 Index strategy

- B-tree: default, equality and range. Use covering indexes (`INCLUDE`) for index-only scans.
- Partial indexes for `WHERE status = 'active'` patterns — smaller and faster.
- BRIN for very large, naturally clustered tables (time-series, log data).
- GIN for arrays, JSONB, full-text. Pair with `fastupdate=off` for read-heavy workloads.
- Drop unused and duplicate indexes — they slow writes and bloat WAL.

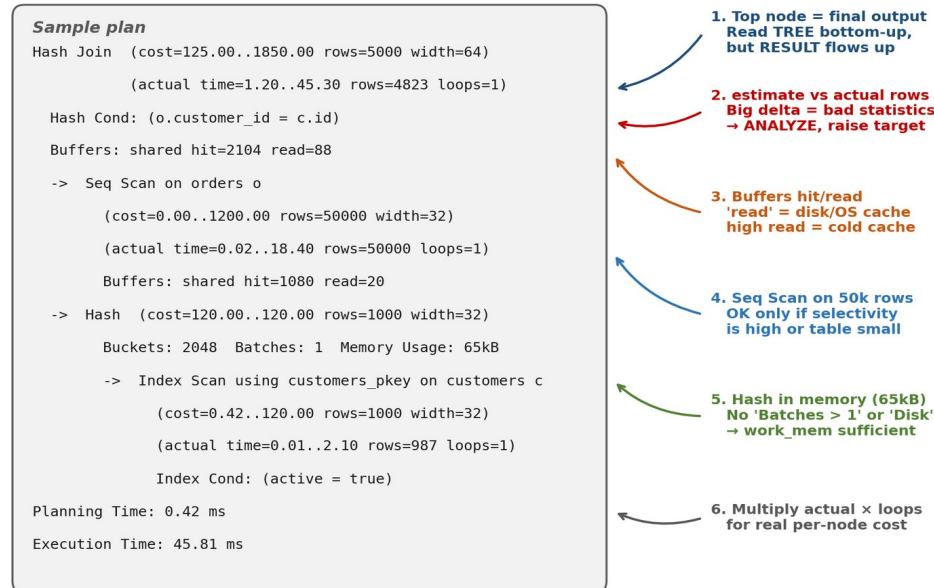
5.5 Frequent anti-patterns

- `SELECT *` — blocks index-only scans and wastes network.
- `OFFSET` on deep pages — use keyset (seek-method) pagination with `WHERE (col, id) > (?, ?)`.
- N+1 query loops from ORMs — batch with `IN (...)` or `ANY($1::int[])`.
- Implicit type casts (`varchar = int`) — silently disables index use.
- Long-running idle-in-transaction sessions — block vacuum, bloat tables.

6. Reading EXPLAIN Plans

Use `EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, WAL)` in production diagnostics. Read the plan bottom-up, inside-out: the innermost node executes first and feeds its parent.

Anatomy of an EXPLAIN (ANALYZE, BUFFERS) Plan



Red flags: • rows estimate off by >10× • Sort ... Disk: • Rows Removed by Filter (huge) • Nested Loop with large outer • Buffers: read >> hit

Figure 5 — Anatomy of an EXPLAIN (ANALYZE, BUFFERS) plan with the six checkpoints to verify on every plan.

6.1 What to look for

Signal	Interpretation
rows= vs actual rows	Order-of-magnitude mismatches mean the planner is flying blind. Fix with ANALYZE, raise statistics target, or add extended statistics.
loops=	Multiply 'actual time' by 'loops' for a node's real cost. A 0.1ms node with 50000 loops is 5 seconds.
Buffers: shared hit/read	'read' = went to disk (or to the OS cache). High read counts on hot tables = undersized shared_buffers or cold cache.
Seq Scan on large table	Acceptable for >20–30% selectivity or small tables. Otherwise check for missing/unused indexes.
Sort ... Disk:	Sort spilled because work_mem was too small. Either raise work_mem for that session or add a supporting index.
Rows Removed by Filter	Post-scan filtering — index isn't covering the predicate. Consider a composite or partial index.
Nested Loop with large outer	Often a row-estimate error. Confirm with actual rows; consider disabling nested loop for the test (SET enable_nestloop = off).

6.2 Visualizers

- explain.depesz.com — color-codes by node cost and shows estimate vs actual deltas.

- explain.dalibo.com — interactive tree, plan history, sharing.
- PEV2 / pev (Postgres Explain Visualizer) — self-hosted alternative for sensitive plans.

Workflow

Capture plans with `auto_explain (log_format=json)`, pipe to a visualizer, and compare plan hashes over time. Plan changes after deploys are the #1 cause of 'the same query is suddenly slow.'

7. AWS RDS / Aurora PostgreSQL

On RDS you do not have OS access or superuser. Tuning happens through parameter groups, RDS-native observability, and the subset of extensions AWS allows.

Cloud-Managed PostgreSQL — Observability Stacks

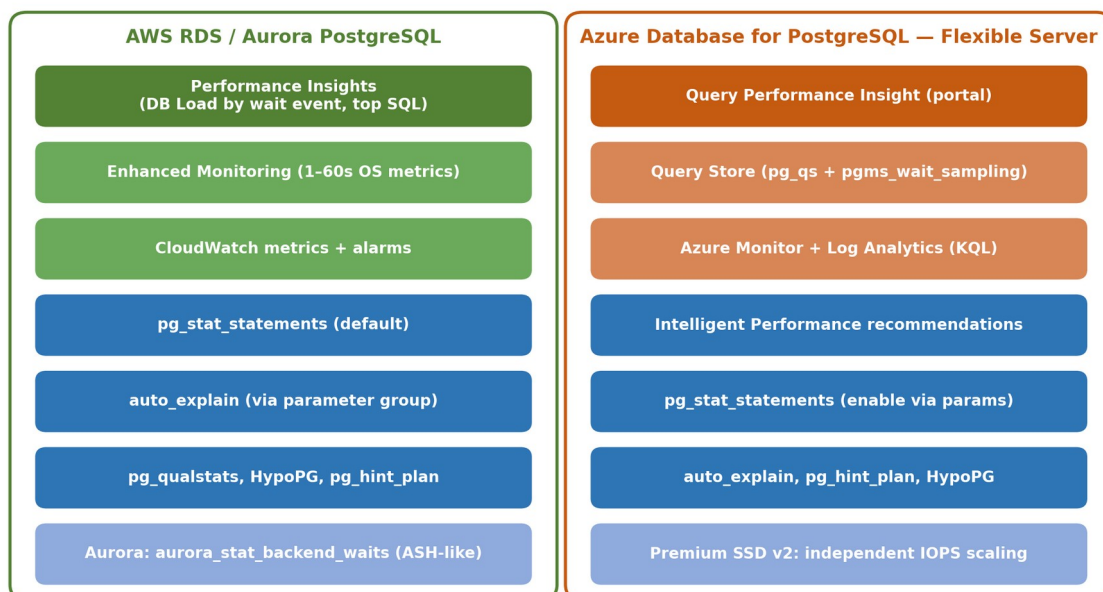


Figure 6 — Observability stacks side by side. Both clouds use `pg_stat_statements` at the bottom but expose different higher-level abstractions.

7.1 Performance Insights

- Visualizes DB Load (Average Active Sessions) sliced by wait event, SQL, user, host, or application.
- The vertical line at **vCPUs** is your saturation threshold — sustained AAS above that line means contention or saturation.
- Wait events to watch: `Lock:transactionid`, `LWLock:BufferContent`, `IO:DataFileRead`, `Client:ClientRead`.
- 7 days of history is free; up to 24 months retention with paid tier.

7.2 Enhanced Monitoring vs CloudWatch

- CloudWatch: hypervisor view, 60s default granularity. Good for alarms (CPU, FreeableMemory, ReadIOPS, WriteIOPS, ReplicaLag, DatabaseConnections).
- Enhanced Monitoring: OS agent view at 1–60s granularity. Surfaces per-process CPU, memory, iowait — invisible to CloudWatch.

7.3 Useful RDS / Aurora extensions

- `pg_stat_statements` — enabled by default in the default parameter group. Verify.
- `auto_explain` — set as a parameter group entry (`shared_preload_libraries` requires reboot).
- `pg_hint_plan`, `pg_qualstats`, HypoPG — available on recent versions.
- Aurora-specific: `aurora_stat_utils`, `aurora_stat_backend_waits` (ASH-equivalent), and Babelfish where T-SQL compatibility matters.

7.4 RDS-specific tuning levers

- Storage: gp3 with provisioned IOPS/throughput beats gp2 in almost all cases; io2 Block Express for latency-sensitive OLTP.
- RDS Optimized Reads (NVMe instance store for temp + sort spill) and Optimized Writes (atomic 16K writes, removes full-page-write tax).
- Use RDS Proxy for connection pooling when application-side pooling (PgBouncer) isn't feasible.
- Aurora: read replicas share storage — replica lag is measured in ms, not GB of WAL. Use the cluster reader endpoint.

8. Azure Database for PostgreSQL — Flexible Server

Flexible Server (the current SKU; Single Server is deprecated) exposes more knobs than RDS, including VM-level metrics and Microsoft-specific Query Store.

8.1 Query Performance Insight & Query Store

- Query Store is implemented by the `pg_qs` extension (Microsoft's port of the SQL Server feature). Captures runtime statistics and wait stats per query, persisted to system tables.
- Enable via server parameters: `pg_qs.query_capture_mode = TOP`, `pgms_wait_sampling.query_capture_mode = ALL`.
- Query Performance Insight (Azure portal) reads from Query Store: top queries by duration, IO, temp file usage, plus waits.

8.2 Intelligent Performance recommendations

- Index recommendations driven by Query Store data — proposed `CREATE INDEX` statements with estimated impact.
- Performance recommendations also surface unused indexes and bloat candidates.

8.3 Tier and IOPS considerations

Tier	Profile	Notes
Burstable (B-series)	Dev/test, intermittent	CPU credits — sustained load exhausts credits and throttles. Avoid for production OLTP.
General Purpose (D-series)	Most OLTP	Balanced vCPU:RAM. Pair with Premium SSD v2 for tunable IOPS/throughput.
Memory Optimized (E-series)	Cache-heavy, analytics	Higher RAM-to-vCPU; helpful when working set fits in shared_buffers + page cache.

8.4 Storage and HA

- Premium SSD v2 lets you scale IOPS and throughput independently of disk size — unlike Premium SSD v1 where IOPS scales with capacity.
- Zone-redundant HA: synchronous standby in another AZ. Expect higher commit latency; tune `synchronous_commit` accordingly for non-critical workloads.
- Azure Monitor + Log Analytics workspace: route server logs (including `auto_explain` output) for KQL queries.

9. Diagnostic Cheat Sheet

9.1 Cache hit ratio (>99% target on OLTP)

```
SELECT datname,
       round(100.0 * sum(blks_hit) / nullif(sum(blks_hit + blks_read), 0), 2)
       AS cache_hit_pct
FROM   pg_stat_database
GROUP BY datname;
```

9.2 Unused indexes (after at least a full workload cycle)

```
SELECT schemaname, relname, indexrelname,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size, idx_scan
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
      AND indexrelid NOT IN (SELECT conindid FROM pg_constraint WHERE conindid > 0)
ORDER BY pg_relation_size(indexrelid) DESC;
```

9.3 Bloat estimate (quick check; use `pgstattuple` for precise)

```
SELECT relname,
       n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / nullif(n_live_tup + n_dead_tup, 0), 1)
       AS dead_pct,
       last_autovacuum
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 1000
ORDER BY dead_pct DESC NULLS LAST
```

```
LIMIT 20;
```

9.4 Long-running and blocked queries

```
SELECT pid, now() - xact_start AS xact_age,  
       state, wait_event_type, wait_event,  
       substring(query, 1, 100) AS query  
FROM pg_stat_activity  
WHERE state <> 'idle'  
      AND xact_start IS NOT NULL  
ORDER BY xact_age DESC;
```

9.5 Top wait events (sampled snapshot)

```
SELECT wait_event_type, wait_event, count(*)  
FROM pg_stat_activity  
WHERE state = 'active'  
GROUP BY 1, 2  
ORDER BY 3 DESC;
```

10. Hands-On Labs

The following labs are designed to be run in order on a dedicated sandbox. Each lab includes setup commands, expected output, and discussion prompts. Plan ~20–30 minutes per lab in a classroom setting.

Sandbox setup

All labs assume PostgreSQL 15+ running locally on Ubuntu/Debian, port 5432, with a 'postgres' superuser and a 'lab' database. Cloud labs (Labs 9 and 10) require an RDS instance and an Azure Flexible Server respectively. Allocate at least 2 vCPU and 4 GB RAM to the sandbox.

Lab 0 · Build the Sandbox

Objective: Create a reproducible workload that every subsequent lab can use.

Prerequisites: PostgreSQL 15+, superuser access, ~2 GB free disk.

Steps

1. Create the lab database and seed it with pgbench at scale factor 50 (~750 MB):

```
createdb lab  
pgbench -i -s 50 -d lab  
  
# Verify  
psql -d lab -c "\dt+"  
# Expect: pgbench_accounts (5M rows, ~640 MB), pgbench_branches, pgbench_history,  
pgbench_tellers
```

2. Enable required extensions and reload:

```
ALTER SYSTEM SET shared_preload_libraries = 'pg_stat_statements, auto_explain';
ALTER SYSTEM SET pg_stat_statements.track = 'all';
ALTER SYSTEM SET auto_explain.log_min_duration = '500ms';
ALTER SYSTEM SET auto_explain.log_analyze = on;
ALTER SYSTEM SET auto_explain.log_buffers = on;
-- Restart required for shared_preload_libraries:
sudo systemctl restart postgresql

-- After restart:
psql -d lab -c "CREATE EXTENSION pg_stat_statements;"
```

3. Generate baseline load (run for at least 60 seconds):

```
pgbench -c 8 -j 2 -T 60 -d lab
# Note the TPS – you'll compare against this number after tuning
```

Discussion / Observe

- Confirm `pg_stat_statements` is populated: `SELECT count(*) FROM pg_stat_statements;`
- Record baseline TPS for later comparison.

Lab 1 · Profile a Workload with `pg_stat_statements`

Objective: Identify the top queries by total cost and by latency, and distinguish the two.

Prerequisites: Lab 0 completed.

Steps

1. Reset statistics to start with a clean window:

```
psql -d lab -c "SELECT pg_stat_statements_reset();"
pgbench -c 8 -T 60 -d lab
```

2. Find top 5 queries by total accumulated time:

```
SELECT substring(query, 1, 60) AS q,
       calls,
       round(total_exec_time::numeric, 0) AS total_ms,
       round(mean_exec_time::numeric, 2) AS mean_ms,
       rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 5;
```

3. Now find top 5 by mean latency (different list!):

```
SELECT substring(query, 1, 60) AS q, calls,
       round(mean_exec_time::numeric, 2) AS mean_ms
FROM pg_stat_statements
WHERE calls > 50 -- exclude noise
ORDER BY mean_exec_time DESC
LIMIT 5;
```

4. Compute cache hit ratio per query:

```
SELECT substring(query, 1, 50) AS q,
       shared_blks_hit, shared_blks_read,
       round(100.0 * shared_blks_hit /
            nullif(shared_blks_hit + shared_blks_read, 0), 1) AS hit_pct
FROM pg_stat_statements
ORDER BY shared_blks_read DESC
LIMIT 10;
```

Discussion / Observe

- Compare the two top-5 lists. A query high in both is the strongest candidate for optimization. A high-mean / low-calls query is a rare but expensive operation — different remediation path.
- Discuss: why does `total_exec_time` usually surface the most-impactful targets, even when individual queries are fast?

Lab 2 · Generate a pgBadger Report

Objective: Turn raw PostgreSQL logs into a shareable HTML performance report.

Prerequisites: Lab 0 completed; pgBadger installed (`sudo apt install pgbadger` or via CPAN).

Steps

1. Configure logging for pgBadger compatibility:

```
ALTER SYSTEM SET log_min_duration_statement = '200ms';
ALTER SYSTEM SET log_line_prefix = '%t [%p]: db=%d,user=%u,app=%a,client=%h ';
ALTER SYSTEM SET log_checkpoints = on;
ALTER SYSTEM SET log_lock_waits = on;
ALTER SYSTEM SET log_temp_files = 0;
ALTER SYSTEM SET log_autovacuum_min_duration = 0;
SELECT pg_reload_conf();
```

2. Generate workload to populate the log:

```
pgbench -c 16 -j 4 -T 120 -d lab
# Mix in some slow queries to ensure variety:
psql -d lab -c "SELECT count(*) FROM pgbench_accounts WHERE filler LIKE '%abc%';"
```

3. Run pgBadger and open the report:

```
mkdir -p ~/pgbadger_reports
pgbadger -j 4 -o ~/pgbadger_reports/ /var/log/postgresql/postgresql-*.log
# Open ~/pgbadger_reports/index.html in a browser
```

Discussion / Observe

- Inspect the 'Top Queries' tab — note time-consuming, slowest individual, and most frequent.
- Check the 'Events' tab for lock waits or checkpoint warnings.
- Variation: run with `--incremental` and `--outdir` to simulate daily report generation.

Lab 3 · Collect PG + OS Stats with pgCluu

Objective: Correlate PostgreSQL metrics with OS-level resource usage in a single report.

Prerequisites: Lab 0 completed; pgCluu installed (`git clone https://github.com/darold/pgcluu && cd pgcluu && make && sudo make install`); sysstat package installed (`sudo apt install sysstat`).

Steps

1. Start the collector for 10 minutes, sampling every 10 seconds:

```
mkdir -p ~/pgcluu_data
pgcluu_collectd -D -i 10 -d lab ~/pgcluu_data
# -D = daemonize, -i = interval in seconds
```

2. Generate workload during the collection window:

```
pgbench -c 12 -j 4 -T 300 -d lab
# In another shell, run a heavy analytical query:
psql -d lab -c "SELECT bid, count(*), avg(abalance) FROM pgbench_accounts GROUP BY bid;"
```

3. Stop the collector and generate the report:

```
pkill -f pgcluu_collectd
pgcluu -o ~/pgcluu_report ~/pgcluu_data
# Open ~/pgcluu_report/index.html
```

Discussion / Observe

- Look at the 'System' tab: do CPU peaks correlate with the pgbench window?
- Compare the 'Database' tab's transaction-per-second graph to the 'System' tab's iowait line — this is the diagnostic value of pgCluu.
- Discuss when this approach beats Prometheus: short-lived diagnostics, no infrastructure required, easy to ship as a tarball.

Lab 4 · Capture Slow Plans Automatically with auto_explain

Objective: Confirm that long-running queries have their plans logged for forensic review.

Prerequisites: Lab 0 completed; auto_explain is loaded via shared_preload_libraries.

Steps

1. Tighten the threshold for the lab:

```
ALTER SYSTEM SET auto_explain.log_min_duration = '100ms';
ALTER SYSTEM SET auto_explain.log_analyze = on;
ALTER SYSTEM SET auto_explain.log_buffers = on;
ALTER SYSTEM SET auto_explain.log_format = 'text';
SELECT pg_reload_conf();
```

2. Trigger a deliberately slow query:

```
psql -d lab -c "SELECT count(*) FROM pgbench_accounts a
                JOIN pgbench_accounts b ON b.aid = a.bid
                WHERE a.filler LIKE '%xy%';"
```

3. Inspect the log:

```
sudo tail -n 80 /var/log/postgresql/postgresql-*.log
# You should see lines starting with 'duration:' followed by the plan tree.
```

4. Set `auto_explain.log_format = 'json'` and re-run for machine-readable plans suitable for `explain.depesz.com`.

Discussion / Observe

- Discussion: in production, why is `sample_rate = 0.05` a more responsible setting than 1.0?
- Discussion: `log_timing = off` trades plan-node timestamps for reduced overhead — when is that the right call?

Lab 5 · Quantify a Parameter Change with pgbench

Objective: Establish a baseline, change one parameter, re-measure, and prove the delta.

Prerequisites: Lab 0 completed.

Steps

1. Baseline run — record TPS:

```
pgbench -c 16 -j 4 -T 60 -d lab -P 5
# -P 5 prints progress every 5s. Note 'tps = X (excluding connections...)'.

```

2. Halve `work_mem` and re-run to provoke a regression:

```
ALTER SYSTEM SET work_mem = '1MB';
SELECT pg_reload_conf();
pgbench -c 16 -j 4 -T 60 -d lab -P 5
-- Expect higher sort-spill activity (check pg_stat_database temp_bytes)

```

3. Restore and increase, re-measure:

```
ALTER SYSTEM SET work_mem = '64MB';
SELECT pg_reload_conf();
pgbench -c 16 -j 4 -T 60 -d lab -P 5

```

4. Try a custom-script benchmark closer to your real workload:

```
cat > /tmp/mixed.sql <<'EOF'
\set aid random(1, 5000000)
BEGIN;
UPDATE pgbench_accounts SET abalance = abalance + 1 WHERE aid = :aid;
SELECT abalance FROM pgbench_accounts WHERE aid = :aid;
COMMIT;
EOF

```

```
pgbench -f /tmp/mixed.sql -c 16 -T 60 -d lab
```

Discussion / Observe

- Document the three TPS numbers and the parameter change between them. This is the basic shape of every tuning proof.
- Discuss why you must run each test for ≥ 60 s — short runs are dominated by cache warmup.

Lab 6 · Read and Fix a Bad EXPLAIN Plan

Objective: Diagnose a plan with row-estimate errors and confirm the fix.

Prerequisites: Lab 0 completed.

Steps

1. Create a skewed table to provoke a bad estimate:

```
CREATE TABLE events (  
    id bigserial PRIMARY KEY,  
    status text NOT NULL,  
    payload text  
);  
INSERT INTO events (status, payload)  
SELECT CASE WHEN i % 1000 = 0 THEN 'error' ELSE 'ok' END,  
        repeat('x', 50)  
FROM generate_series(1, 1000000) i;  
CREATE INDEX events_status_idx ON events(status);  
-- DO NOT ANALYZE yet.
```

2. Look at the plan with stale statistics:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM events WHERE status = 'error';  
-- Note the 'rows=' estimate vs 'actual rows='
```

3. Refresh statistics and re-run:

```
ANALYZE events;  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM events WHERE status = 'error';  
-- Estimate should now be close to actual; plan may change.
```

4. Now provoke the opposite: a function on the indexed column:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM events WHERE lower(status) = 'error';  
-- Seq Scan! Predicate is not sargable.  
  
-- Fix with a functional index OR rewrite the predicate:  
CREATE INDEX events_lower_status ON events (lower(status));  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM events WHERE lower(status) = 'error';
```

Discussion / Observe

- Capture the three plans and paste them into explain.depesz.com side by side.
- Identify the exact node where Buffers 'read' count drops between the un-analyzed and analyzed runs.
- Discuss: which fix carries less ongoing maintenance cost — the functional index or the rewritten predicate?

Lab 7 · Test Indexes Before Creating Them with HypoPG

Objective: Use hypothetical indexes to validate a tuning hypothesis without paying the CREATE INDEX cost.

Prerequisites: Lab 0 completed; HypoPG installed (`sudo apt install postgresql-15-hypopg`).

Steps

1. Install and explore:

```
CREATE EXTENSION hypopg;

-- A query that currently does a Seq Scan
EXPLAIN SELECT * FROM pgbench_accounts WHERE filler LIKE '%abc%';
```

2. Create a hypothetical index and re-EXPLAIN:

```
SELECT * FROM hypopg_create_index(
    'CREATE INDEX ON pgbench_accounts (filler text_pattern_ops)');

EXPLAIN SELECT * FROM pgbench_accounts WHERE filler LIKE '%abc%';
-- The planner may or may not pick it – LIKE '%...%' is not sargable for B-tree.
```

3. Try a more realistic case — a missing index on a join column:

```
EXPLAIN ANALYZE
SELECT a.aid, h.delta
  FROM pgbench_accounts a
     JOIN pgbench_history h ON h.aid = a.aid
 WHERE a.bid = 5;

SELECT * FROM hypopg_create_index('CREATE INDEX ON pgbench_history(aid)');
EXPLAIN SELECT a.aid, h.delta FROM pgbench_accounts a
     JOIN pgbench_history h ON h.aid = a.aid WHERE a.bid = 5;

-- Clean up:
SELECT hypopg_reset();
```

Discussion / Observe

- Note that EXPLAIN ANALYZE does NOT work with hypothetical indexes — only EXPLAIN. Why?
- Discuss how this fits an index-design review: propose, simulate, ship. No production-impact test required.

Lab 8 · Find Missing Indexes with pg_qualstats and PoWA

Objective: Use predicate statistics to surface indexes that pg_stat_statements alone cannot suggest.

Prerequisites: Lab 0 completed; pg_qualstats and PoWA installed (`sudo apt install postgresql-15-pg-qualstats postgresql-15-powa`).

Steps

1. Add extensions to preload list and restart:

```
ALTER SYSTEM SET shared_preload_libraries =  
    'pg_stat_statements, pg_qualstats, pg_stat_kcache, auto_explain';  
sudo systemctl restart postgresql  
  
psql -d lab -c "CREATE EXTENSION pg_qualstats;  
                CREATE EXTENSION pg_stat_kcache;  
                CREATE EXTENSION powa;"
```

2. Generate diverse load with predicates the planner sees:

```
pgbench -c 8 -T 120 -d lab  
psql -d lab -c "SELECT * FROM pgbench_accounts WHERE bid BETWEEN 10 AND 20;"  
psql -d lab -c "SELECT * FROM pgbench_history WHERE delta > 100;"
```

3. Query pg_qualstats for the most-evaluated predicates:

```
SELECT to_regclass(c.oid)::text AS relation,  
       a.attname AS column,  
       q.occurrences,  
       q.execution_count,  
       q.nbfiltered  
FROM pg_qualstats() q  
JOIN pg_class c ON c.oid = q.lrelid  
JOIN pg_attribute a ON a.attrelid = c.oid AND a.attnum = q.lattnum  
ORDER BY q.nbfiltered DESC  
LIMIT 10;
```

4. Optional: install powa-web and browse the index advisor at <http://localhost:8888>.

Discussion / Observe

- Note columns with high 'nbfiltered' — predicates that scanned rows but rejected most of them. Prime index candidates.
- Combine with HypoPG (Lab 7) to validate before issuing CREATE INDEX.

Lab 9 · Diagnose Load on AWS RDS with Performance Insights

Objective: Translate 'the database is slow' into a wait-event answer using RDS-native tooling.

Prerequisites: RDS PostgreSQL 15+ instance with Performance Insights enabled (default for db.r5.large+).

Steps

1. From the AWS console, open RDS → Databases → <your DB> → Monitoring → Performance Insights.

2. Generate sustained load against the RDS endpoint from an EC2 host in the same VPC:

```
pgbench -i -s 100 -h <rds-endpoint> -U <user> -d lab
pgbench -c 50 -j 4 -T 600 -h <rds-endpoint> -U <user> -d lab
```

3. In the Performance Insights chart, switch 'Slice by' to:
 - Wait — see whether Lock, IO:DataFileRead, or Client:ClientRead dominates.
 - SQL — see which normalized statement is consuming DB load.
 - User / Host / Application — confirm the source is your pgbench client.
4. Compare the AAS (Average Active Sessions) line against the vCPU saturation line — sustained AAS above vCPU is the diagnostic signal of CPU/lock contention.
5. Cross-reference with Enhanced Monitoring (1s granularity) to confirm OS-level pressure (iowait vs user CPU).

Discussion / Observe

- Discuss: Client:ClientRead near the top is almost never a database problem — what does it mean?
- Note that Performance Insights persists 7 days free — set a CloudWatch alarm on DBLoad to alert before the user does.

Lab 10 · Azure Flexible Server — Query Store & Performance Insight

Objective: Use the Microsoft Query Store extension to identify regressions and resource hogs.

Prerequisites: Azure Database for PostgreSQL Flexible Server (General Purpose tier).

Steps

1. Enable Query Store via server parameters (Azure portal → Server parameters):

```
pg_qs.query_capture_mode = TOP
pgms_wait_sampling.query_capture_mode = ALL
pg_qs.retention_period_in_days = 7
pg_qs.store_query_plans = on
```

2. Generate workload from an Azure VM or Cloud Shell in the same region:

```
pgbench -i -s 50 -h <server>.postgres.database.azure.com -U <admin> -d lab
pgbench -c 30 -j 4 -T 300 -h <server>.postgres.database.azure.com -U <admin> -d lab
```

3. Open Server → Query Performance Insight in the portal. Inspect:
 - Long-running queries tab — top queries by mean duration.
 - Wait statistics — top wait categories over the window.
 - Resource consuming queries — by CPU, IO, temp file usage.
4. Query the Query Store tables directly for analyses the portal does not surface:

```
-- Connect to the azure_sys database
\c azure_sys

SELECT query_text_id, substring(query_sql_text, 1, 60) AS q,
```

```
sum(execution_count) AS calls,  
round(avg(mean_time)::numeric, 2) AS avg_ms  
FROM query_store.qs_view  
WHERE start_time > now() - interval '1 hour'  
GROUP BY 1, 2  
ORDER BY sum(execution_count * mean_time) DESC  
LIMIT 10;
```

5. Review Intelligent Performance recommendations in the portal — index suggestions are derived from this same data.

Discussion / Observe

- Compare the Microsoft `pg_qs` model to the open-source PoWA + `pg_stat_statements` stack — what do you gain, what do you lose?
- Discuss the privacy implication of storing normalized query text in a system database; Query Store can be disabled per-server when this is sensitive.

11. Reference Order of Operations

1. Enable `pg_stat_statements`, `auto_explain`, and detailed logging (`log_min_duration_statement`, `log_lock_waits`, `log_temp_files`).
2. Establish a workload baseline — TPS, AAS, top-N queries, wait profile.
3. Capture the worst offenders' plans with EXPLAIN (ANALYZE, BUFFERS) and validate row estimates.
4. Fix at the highest level that resolves the issue: query rewrite > index > statistics > parameter > hardware.
5. Re-measure. Keep snapshots so you can prove the delta.

Training note

Teach this in the order: workload → query → plan → server → OS. Students who jump to `shared_buffers` before understanding `pg_stat_statements` end up tuning the wrong layer.